

16-720 Computer Vision

Assignment 2: Representations and Visual Recognition

Fall 2026

Released September 29, 2026

This handout describes the assignment requirements and policies. The README covers setup, and the starter notebook provides code and examples. Read the full handout before starting.

Item	Requirement
Release	September 29, 2026
Due	October 27, 2026 (Gradescope)
Base score	100 points
Optional credit	Up to 10 points, including showcase participation
Expected time	12+ hours
Runtime	Google Colab
Submit	Code, completed notebook, L ^A T _E X-generated PDF, checkpoint, Recognition Atlas, Hero Result

1. Overview

In this assignment, you will move from classical image representations to learned ones. You will derive the mathematics behind softmax classification and residual learning, implement and train a compact residual convolutional network, inspect the features it learns, and compare its behavior with simpler baselines. Your final goal is to create a **Recognition Atlas**: a polished, self-contained visual artifact that teaches an audience how a recognition system sees a small collection of categories that you choose. The atlas must show what the system recognizes, where it succeeds or struggles, what nearby images look like in feature space, and evidence about an informative failure or boundary case.

2. Learning objectives

After completing this assignment, you should be able to:

- derive and interpret the softmax cross-entropy gradient;
- calculate convolutional output sizes, parameter counts, and receptive fields;
- explain how residual connections alter information and gradient flow;
- implement, train, and evaluate a compact convolutional recognition system;
- extract and compare raw, hand-designed, and learned image representations;
- diagnose classification errors, shortcut learning, and sensitivity to nuisance changes;
- use a pretrained recognition model responsibly inside a larger visual application; and
- communicate model behavior through quantitative evidence and clear visual design.

3. Academic-integrity and generative-AI policy

Work	Policy
Part A	Generative AI is NOT permitted in any form.
Part B	Generative AI is NOT permitted in any form.
Part C	Generative AI is permitted with disclosure.
Part D	Generative AI is permitted and encouraged with disclosure, but it may not replace or modify the required Part B implementation.
Part E	Generative AI is permitted with disclosure.

The policy is defined by the work being completed, not by the file or chat in which it occurs. For Part A and Part B, you may use course notes, textbooks, papers, conventional web resources, official NumPy, PyTorch, and torchvision documentation, and course staff as allowed by the collaboration policy. Replacing the required model or TODO implementa-

tions with a ready-made system does not satisfy the implementation requirements. Generative AI is never required. If you do not use it in a permitted section, say so and complete the no-AI alternative in D4. In all cases, you are responsible for understanding, testing, and disclosing what you submit.

Working with others. Talk to your classmates. Discussing the derivations, debugging strategies, and experimental design with other students is one of the best ways to learn this material, and we encourage it. Everything you submit, including derivations, code, figures, and text, must be produced by you, subject to the permitted and disclosed AI assistance above. You may not read or copy another student's solutions or let anyone copy yours. List everyone you worked with in your acknowledgments; collaboration is never penalized, but unacknowledged copying is an integrity violation.

4. Shared conventions and allowed tools

- Use the released dataset loaders, normalization and augmentation pipeline, fixed training/validation/test splits, training configuration, and supplied infrastructure. The reduced CPU mode is a smoke test; the required training uses the released configuration.
- Follow the model and interface contracts in B2–B4 and the released student docstrings. Run the supplied public tests for shapes, parameter count, skip paths, confusion matrices, and retrieval. Also verify gradient flow and reproducibility; the public tests are not exhaustive.
- Use NumPy, PyTorch, and the supplied helpers for the required TODOs. Do not replace the requested TinyResNet with a torchvision model or another ready-made network.
- Course-approved pretrained CNNs or vision transformers are permitted in Part D as frozen feature extractors. Keep that application distinct from the required Part B model and results; Part E extensions must also be clearly separated.
- For retrieval, use the released distance or similarity measure, exclude the query itself when appropriate, and return deterministic sorted indices and scores, as specified in B4.

Every graded figure and table must appear both as a saved output in the executable notebook and in the matching labeled section of the written PDF. The full Recognition Atlas is submitted separately with its static Hero Result preview; it does not replace the required written evidence.

5. Part A — THINK: Scores, Features, and Residual Learning (25 points; 1.5–2 hours)**5.1 A1 — Softmax cross-entropy from first principles (9 points)**

A classifier produces a score vector $\mathbf{s} \in \mathbb{R}^C$ for one image. Its softmax probabilities and cross-entropy loss for true class y :

$$p_c = \frac{e^{s_c}}{\sum_{j=1}^C e^{s_j}}, \quad \mathcal{L}(\mathbf{s}, y) = -\log p_y.$$

1. Rewrite $\mathcal{L}$ as a function of the logits $\mathbf{s}$ without using p_y .
2. Derive $\partial \mathcal{L} / \partial s_k$ separately for $k = y$ and $k \neq y$.
3. Combine the cases to show

$$\frac{\partial \mathcal{L}}{\partial s_k} = p_k - \mathbf{1}[k = y].$$

4. Explain why increasing the correct-class logit reduces the loss, and why the size of the update depends on the model's confidence.
5. Show that adding the same scalar a to every logit leaves the softmax probabilities unchanged. Explain why the numerically stable implementation subtracts $\max_j s_j$ before exponentiation.

5.2 A2 — Work through one prediction by hand (5 points)

For logits $\mathbf{s} = [\ln 2, \ln 3, 0]$ and true class $y = 2$ using one-indexed class labels:

1. compute the three softmax probabilities exactly;
2. compute the cross-entropy loss;
3. compute the gradient with respect to all three logits; and
4. take one gradient-descent step with learning rate $\eta = 0.5$ and state whether the correct-class probability should increase or decrease. You do not need to exponentiate the updated logits.

5.3 A3 — Convolutional accounting and receptive fields (6 points)

Consider the following network. Every convolution and the linear layer use a bias. Global average pooling keeps singleton spatial dimensions; flatten before the linear layer. Spatial dimensions are height by width.

Layer	Kernel	Stride	Padding	Output channels
Input	—	—	—	3 channels, 32×32
Conv 1	3×3	1	1	16
Conv 2	3×3	2	1	32
Conv 3	3×3	1	1	32
Global average pool	—	—	—	32
Linear	—	—	—	10

1. Compute the output shape after every layer for a minibatch of size B .
2. Compute the number of learned parameters in each convolution and in the final linear layer.
3. Compute the receptive-field size and effective stride after Conv 1, Conv 2, and Conv 3. Begin with receptive field $r_0 = 1$ and jump $j_0 = 1$, and use

$$r_\ell = r_{\ell-1} + (k_\ell - 1)j_{\ell-1}, \quad j_\ell = j_{\ell-1}s_\ell.$$

4. Explain one advantage of replacing a large fully connected spatial layer with global average pooling.

5.4 A4 — Why residual connections help (5 points)

First consider a simplified identity-skip residual block without a post-addition activation:

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x}; \theta),$$

and a later network produces scalar loss $\mathcal{J}(\mathbf{y})$.

1. Use the chain rule to derive $\partial \mathcal{J} / \partial \mathbf{x}$ in terms of $\partial \mathcal{J} / \partial \mathbf{y}$ and the Jacobian of F .
2. Identify the term that provides a direct gradient path.
3. Explain why setting the parameters of F so that $F(\mathbf{x}) \approx \mathbf{0}$ makes the block close to an identity mapping. How does this conclusion change for B2's post-addition ReLU when $\mathbf{x}$ has negative entries?
4. Predict how a residual block and a plain two-layer block may differ when additional layers are unnecessary. Revisit this prediction in Part C.

6. Part B — BUILD: Train a Compact Recognition System (35 points; about 3 hours)

You may use NumPy, PyTorch, and torchvision documentation and course materials. Starter code provides dataset loading, deterministic training/validation/test splits, plotting, the training configuration, checkpointing, and Colab setup. The required dataset is the released CIFAR-10 subset: 25,000 training and 5,000 validation images, balanced across all ten classes, plus the full 10,000-image test set. Inputs are 32×32 RGB. Standard mode uses seed 16720, batch size 128, 25 epochs, and the supplied AdamW/cosine-schedule configuration. Use the supplied transforms and split indices.

6.1 B1 — Read the pipeline (5 points)

Before implementing anything, inspect the starter notebook and answer its five labeled questions, **B1-Q1–B1-Q5**, in the corresponding template slots. They cover shapes, transforms, BatchNorm modes, logits/loss/confidence, and feature extraction. For functions that are still TODOs, describe the required behavior from the docstrings and B2–B4; you are not expected to locate implementation lines that do not yet exist.

6.2 B2 — Residual classifier (14 points)

Implement the essential model components in the supplied TODOs:

- `ConvNormAct(in_ch, out_ch, stride)`: a 3×3 convolution, normalization, and ReLU in the released order;
- `ResidualBlock(in_ch, out_ch, stride)`: two convolutional stages plus a skip path that handles both identity and shape-changing cases;
- `TinyResNet(num_classes)`: a stem, three residual stages, global average pooling, and a linear classifier; and
- `forward_features(x)`: return the final spatial feature map and pooled embedding before classification.

Use the following architecture; do not replace it with a torchvision model. All 3×3 convolutions use padding 1 and no bias. Normalization is `BatchNorm2d`; the final linear classifier uses a bias.

Module	Output channels	Blocks	First stride
stem (ConvNormAct)	32	—	1
stage1	32	2	1
stage2	64	2	2
stage3	128	2	2

Only the first block of a stage may downsample; all second blocks use stride 1. Finish with `pool` (adaptive average pooling to 1×1), flattening, and `classifier` (128 to `num_classes`). Preserve these module names. In `ConvNormAct`, register `conv`, `norm`, and `act` in that order; name the skip module `skip`. These names are used by the tests and visualization helpers. Run the public tests and verify gradient flow and reproducibility.

Produce a compact architecture trace containing the shape after each stage and the total number of learned parameters. Visualize at least eight first-layer filters and feature maps from two different stages for one test image. The supplied B2 visualization runs before training; label it as untrained. Label any post-training replacement with its checkpoint.

Residual block contract. The main branch is 3×3 convolution (block stride), BatchNorm, ReLU, then 3×3 convolution (stride 1) and BatchNorm, with no ReLU before addition. If shapes agree, `skip` is `nn.Identity`; otherwise use a bias-free 1×1 convolution with the block stride, followed by BatchNorm. Add equal-shaped branches, then apply ReLU. An identity skip does not make the whole post-activation block an identity for arbitrary signed inputs.

6.3 B3 — Training and evaluation (9 points)

Complete the required training and evaluation functions:

- `train_one_epoch(model, loader, optimizer, device)` must set the correct model mode, clear gradients, compute logits and loss, backpropagate, update parameters, and accumulate sample-weighted metrics;
- `evaluate(model, loader, device)` must avoid gradient tracking and return loss, top-1 accuracy, predictions, labels, and confidence scores; and
- `compute_confusion_matrix(labels, predictions, num_classes)` must return a correctly oriented integer matrix and handle absent classes.

Train the released TinyResNet configuration and plot training/validation loss and accuracy. Evaluate the saved checkpoint with the highest validation accuracy (the earliest epoch on a tie), and report its test accuracy and row-normalized confusion matrix. Loss and accuracy are sample-weighted; return accuracy in $[0, 1]$ and confidence as the maximum softmax probability. Confusion-matrix rows are true classes and columns are predictions; keep absent-class rows zero, including after normalization.

6.4 B4 — Representations and nearest neighbors (7 points)

Implement `extract_embeddings(model, loader, device)` and `nearest_neighbors(query, gallery, k, *, exclude_index=None)`. Extract pooled CPU embeddings and labels in loader order, in evaluation mode without gradients. Retrieval must L2-normalize internally, sort by descending cosine similarity, and break ties by ascending gallery index. For a query drawn from the gallery, pass its original index as `exclude_index`; exclude that entry, not an arbitrary tied match.

Using at least three fixed query images from the test set, show the five nearest test images under each representation. When a query is itself part of the test-image gallery, exclude that same image from its retrieved neighbors.

1. normalized raw pixels: inverse-normalize to RGB in $[0, 1]$, flatten, then L2-normalize;
2. pooled embeddings from an untrained network; and
3. pooled embeddings from your trained network.

Briefly annotate what visual property appears to control each neighborhood.

7. Part C — UNDERSTAND: What Did the Model Learn? (15 points; 1.5–2 hours)

Accuracy alone does not establish understanding. In this section, analyze the learned representation and determine which changes it treats as meaningful.

7.1 C1 — Controlled representation comparison (6 points)

Compare the trained TinyResNet with two supplied baselines:

- the supplied histogram-of-oriented-gradients (HOG) representation; and
- the same TinyResNet architecture with random, untrained weights.

For all three representations, use the supplied deterministic cosine 5-nearest-neighbor classifier to measure classification performance. For this classification experiment, use the training images as the labeled gallery and classify each test image from its five nearest training-image neighbors. Report test accuracy and per-class recall. This is separate from B3's classifier-head accuracy. Separately, create one compact nearest-neighbor visualization comparing all three representations. For this visualization only, reuse B4's fixed test query indices and the test-image gallery, excluding each query image itself from its own retrieved neighbors. Include HOG instead of raw pixels; B4's unchanged panel alone does not satisfy C1. Explain one case where the learned representation improves semantic grouping and one case where the hand-designed or untrained representation exposes a useful low-level similarity.

7.2 C2 — Nuisance and shortcut stress tests (5 points)

Evaluate the trained model under **three** controlled changes chosen from brightness shift, blur, additive noise, partial occlusion, crop/translation, color removal, or background replacement. The notebook defaults to brightness, blur, and central occlusion; the other listed choices are approved alternatives. At least one transformation must preserve the intended class while substantially changing the pixels.

For each condition, use four severity levels including the unmodified baseline, and report accuracy or mean maximum-softmax confidence as a function of severity. Use the same fixed test examples across conditions and report the sample count (the starter uses 32); these are subset results, not full-test accuracy. Apply transformations in RGB before normalization. Show representative successes and failures, or state when none occur. State your prediction before each experiment. Explain the result using receptive fields, augmentation, learned features, or another idea from Parts A or B.

7.3 C3 — Residual connection and failure diagnosis (4 points)

Use the supplied plain-network ablation with approximately matched depth and width. Train fresh plain and residual models for five epochs each in standard mode using the supplied data-stream resets and matched optimizer settings. Retain the printed reset evidence. This controlled comparison is separate from B3's 25-epoch run; the error analysis below may use the B3 checkpoint.

1. Compare early training behavior, final validation performance, or gradient norms with your residual model.
2. Revisit your prediction from A4.
3. Select one confident error from either model. Show the image, predicted and true classes, confidence, nearest embedding neighbors, and the supplied classifier-weight CAM attribution visualization. Give the most plausible explanation, while clearly distinguishing evidence from speculation.

8. Part D — CREATE: Build a Recognition Atlas (25 points; 2.5–3 hours)

8.1 D1 — Choose and document a visual world (4 points)

Choose **three visually related categories** that can be photographed safely and without exposing private information. Examples include three kinds of mugs, shoes, leaves, packaged foods, tools, signs, toys, or architectural details. Collect or create 8–15 images per category. At least half of the images **in each category** must be your own photographs, unless you receive approval for the capture alternative in Section 9. Generated images do not count as own photographs. Each category must include both development and held-out images.

Document the category definitions, source of every image, and one anticipated ambiguity in `atlas_manifest.csv` and your report. Record the split before fitting or selecting a model. Do not reuse the same image under multiple file-names; keep augmented or near-duplicate views of an original in the same split. Do not photograph people without consent, private documents, screens containing personal information, or situations where photography is unsafe or prohibited.

8.2 D2 — Adapt a pretrained representation (7 points)

Use one course-approved pretrained CNN or vision transformer as a frozen feature extractor: torchvision ResNet-18 or Swin-T with the default weights listed in `data/PROVENANCE.md`. Record the exact weight enum and use its preprocessing. Train a lightweight classifier on your categories, or use a clearly justified nearest-neighbor/prototype rule. The supplied end-to-end path uses ResNet-18; Swin-T requires compatible evidence visualization rather than the ResNet-specific CAM helper.

You must make at least one substantive design decision or modification, such as selecting layers, comparing CNN and ViT embeddings, choosing augmentation, calibrating a rejection threshold, balancing classes, or revising the category definitions after inspecting development-set failures. Use only development data to make these choices, and freeze them before evaluating the held-out set. Report development/held-out counts and held-out accuracy. Any further change selected after inspecting held-out results must be labeled exploratory, not an unbiased improvement on unseen data.

8.3 D3 — Recognition Atlas and Hero Result (11 points)

Create one polished visual artifact as either a single-page webpage, a scrollable notebook report exported to HTML, or a concise narrated video no longer than 60 seconds. It must be understandable without reading your code and include:

- the three categories and representative examples;
- predictions and confidence or clearly labeled similarity scores for held-out images; prototype cosine scores are not probabilities;
- a feature-space view or nearest-neighbor gallery;
- one clear success and one informative failure; if all held-out predictions are correct, use a clearly labeled lowest-score boundary case instead of a failure; if none are correct, report that no success was observed;
- evidence about the failure or boundary case, such as attribution, occlusion, retrieval neighbors, or a controlled edit;
- one before/after intervention that improves behavior or clarifies the limitation; declare the intervention before evaluating it, and identify any held-out-informed choice as exploratory; and
- a short, honest statement of what the result does *not* establish.

The D2 design decision may also serve as the D3 intervention when the before/after evidence is included. HTML submissions must work offline with their assets embedded or bundled.

Hero Result. Select the single panel, animation, or interaction that best communicates how your system sees the chosen visual world. It should remain legible outside the notebook and be something you would be comfortable showing in a portfolio or course showcase.

8.4 D4 — Comprehension reflection (3 points)

Write approximately 150–250 words and use one of the following paths.

If you used generative AI in a permitted section:

1. What did generative AI help you implement or communicate?
2. How does the most important AI-assisted portion work?
3. What did you change, debug, test, or redesign from the initially generated version, and why?

If you did not use generative AI:

1. State that no generative AI was used.

2. Explain how the most important Part D portion works.
3. Describe a concrete change, debugging step, or test that improved the result.

The reflection is part of the written PDF. Also include the acknowledgments and disclosure described in Section 11; there is no separate disclosure form.

9. Capture access, privacy, safety, and accessibility

Do not record non-consenting people, private documents, personal information on screens, or unsafe/prohibited locations. Do not take risks to obtain a more dramatic result. Document the source of every image in your Recognition Atlas, as required in D1, and do not submit data containing personal information.

For access, privacy, safety, or accessibility reasons, request a capture alternative through the course staff help channel or office hours before relying on it. With staff approval, alternative images may replace the own-photograph requirement in D1. Record the approval and image sources, and complete the same category documentation, held-out evaluation, Recognition Atlas, Hero Result, and reflection. The technical requirements and grading criteria remain the same. Do not assume that an alternative image collection has been approved without contacting staff.

10. Part E — EXPLORE: Optional extension (up to +10 points; about 1 hour)

Do something we did not explicitly ask you to do. Extend the method, improve it or apply it somewhere unexpected. Work already used to satisfy D2 or D3 does not by itself earn Part E credit; the extension must add a distinct question, experiment, capability, or analysis. Possibilities include:

- compare CNN and ViT attention or attribution behavior;
- use DETR to build a category-aware scene inventory;
- visualize or manipulate a VAE latent space;
- compare a VAE reconstruction with a GAN sample or another generator;
- implement calibration, abstention, or out-of-distribution rejection;
- construct and analyze a universal shortcut or adversarial pattern; or
- improve the atlas into a genuinely interactive teaching tool.

+5 Meaningful additional exploration with a clear result and short explanation.

+10 Particularly thoughtful, technically interesting, creative, or ambitious exploration with convincing evidence.

Part E and showcase participation share one **10-point total extra-credit cap**.

Optional course showcase. Every student must create a Hero Result, but public participation is optional. Students who opt in may post their final artifact to the course Slack `#showcase-hw2` channel within 24 hours after the assignment deadline. Voting uses the designated `:movie_camera:` reaction during a common 72-hour window announced by course staff. Participation earns +1 within the shared cap; vote count never determines academic credit. The course may recognize a People's Choice result and a non-credit Technical Highlight. Selected atlases may be shown briefly in class. Public posting requires the student's affirmative choice.

11. Submission specification

Submit through Gradescope by October 27, 2026:

1. the completed code with all required implementations: `pa2_<andrewid>.py`;
2. the completed, executed starter notebook, with evidence outputs retained: `pa2_<andrewid>.ipynb`;
3. one $\text{\LaTeX}$ -generated written PDF from the provided answer template, including all responses, D4, and acknowledgments/disclosure: `pa2_written_<andrewid>.pdf`;
4. the best-validation TinyResNet checkpoint with its supplied metadata: `tinyresnet_<andrewid>.pt`;
5. all required plots, tables, and visualizations, plus `recognition_atlas_<andrewid>.html` or `.mp4`;
6. the static Hero Result preview: `hero_<andrewid>.png`; and
7. the completed image/split manifest: `atlas_manifest_<andrewid>.csv`.

Replace `<andrewid>` with your Andrew ID. Rename generic notebook exports accordingly before submission. The Atlas bundle is limited to 25 MB and the Hero preview to 10 MB.

Evidence and artifacts. Save figures under `artifacts/` and insert them into the provided $\text{\LaTeX}$ template; displaying a plot alone does not save it. Every required figure and table must appear both in the executable notebook and in the corresponding PDF section. Back up edited code, outputs, and images before disconnecting a Colab runtime. Keep original images and experimental data locally until grades are final. Submit the requested TinyResNet checkpoint, but not pretrained-model caches or raw private photographs. Strip EXIF metadata from submitted image copies.

Submit the Recognition Atlas in one of the formats allowed in D3: a single-page webpage, a notebook report exported to HTML, or a narrated video no longer than 60 seconds. Include the static Hero Result preview even when the atlas is interactive or a video. Public showcase participation is not required for submission.

Written PDF and Gradescope mapping. Use the stable A1–D4 labels in the provided template; include Part E if completed. Do not retype the questions or change their labels. Begin each labeled subproblem on a new page using the template’s page breaks, in template order. After upload, assign every PDF page containing part of an answer to that question in Gradescope and inspect the rendered preview. Do not insert unrelated cover or scratch pages.

Acknowledgments and disclosure. Include a final labeled acknowledgments-and-disclosure section in the written PDF; there is no separate form. List collaborators, outside resources, image and dataset sources, and pretrained weights. State whether you used generative AI. If so, identify the tool or tools, the permitted section or task assisted, and how you used the assistance; explain your understanding and verification in D4. If not, explicitly state that no generative AI was used and complete the no-AI D4 path.

Runtime and privacy. All required code must execute in a fresh Google Colab runtime without paid compute, private services, or credentials. Document the Part D local-image upload step and how its inputs match the submitted manifest. Follow the released training configuration; the CPU smoke-test mode does not replace required training. Do not submit a private API key, authentication token, or dataset containing personal information.

12. Point summary

Part	Subproblem points	Total
A — THINK	A1: 9; A2: 5; A3: 6; A4: 5	25
B — BUILD	B1: 5; B2: 14; B3: 9; B4: 7	35
C — UNDERSTAND	C1: 6; C2: 5; C3: 4	15
D — CREATE	D1: 4; D2: 7; D3: 11; D4: 3	25
Base total		100
E and showcase	Shared extra-credit cap	0 to +10